

```

import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import folium
import plotly.express as px

from streamlit_folium import st_folium
from reportlab.platypus import (
    SimpleDocTemplate,
    Paragraph,
    Spacer,
    Table,
    TableStyle,
    Image
)
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4

# =====
# CONFIG STREAMLIT
# =====

st.set_page_config(
    page_title="EA Comercial Intelligence",
    page_icon="🏢",
    layout="wide"
)
# =====
# VISUAL EA COMERCIAL INTELLIGENCE
# =====

st.markdown(
    """
    <style>

    #MainMenu {visibility:hidden;}
    footer {visibility:hidden;}
    header {visibility:hidden;}

    .stApp{
        background:#f4f6f9;
    }

    .block-container{
        padding-top:1rem;
        padding-bottom:2rem;
        max-width:1450px;
    }

    html, body, [class*="css"]{
        font-family: 'Segoe UI', sans-serif;
    }

    /* SIDEBAR */

    section[data-testid="stSidebar"]{
        background: linear-gradient(180deg,#020617 0%, #071226 100%);
        border-right:1px solid rgba(255,255,255,0.06);
    }

    section[data-testid="stSidebar"] *{
        color:white;
    }

    /* HERO */

    .ea-hero{
        background:
            linear-gradient(
                135deg,
                #c40000 0%,
                #930000 45%,
                #1a0000 100%
            );

        border-radius:22px;

        padding:34px;

        display:flex;
        justify-content:space-between;
        align-items:center;

        box-shadow:
            0px 10px 35px rgba(0,0,0,0.18);

        margin-bottom:28px;

        color:white;
    }

    .ea-hero-left h1{

```

```

        color:white;
        margin:0;
        font-size:52px;
        font-weight:900;
        letter-spacing:-2px;
    }

    .ea-hero-left p{
        margin-top:6px;
        font-size:17px;
        color:#f3f4f6;
    }

    .ea-pill{
        display:inline-block;

        margin-top:18px;

        background:rgba(255,255,255,0.12);

        padding:10px 16px;

        border-radius:999px;

        border:1px solid rgba(255,255,255,0.12);

        font-size:14px;
    }

    .ea-hero-right{
        display:flex;
        gap:18px;
        flex-wrap:wrap;
    }

    .ea-status-card{

        background:rgba(255,255,255,0.08);

        border:1px solid rgba(255,255,255,0.14);

        border-radius:18px;

        padding:18px;

        min-width:220px;

        backdrop-filter: blur(10px);
    }

    .ea-status-card h4{
        margin:0;
        color:white;
        font-size:16px;
    }

    .ea-status-card p{
        margin-top:8px;
        color:#f3f4f6;
        font-size:14px;
    }

    .ea-dot{
        width:12px;
        height:12px;
        border-radius:999px;
        background:#22c55e;
        margin-bottom:10px;
    }

    /* KPI */

    div[data-testid="stMetric"]{

        background:white;

        border-radius:20px;

        padding:24px;

        border:none;

        box-shadow:
        0px 4px 18px rgba(15,23,42,0.06);

        transition:0.2s;
    }

    div[data-testid="stMetric"]:hover{
        transform:translateY(-3px);
    }

    div[data-testid="stMetricLabel"]{

```

```
font-size:15px;
font-weight:600;
color:#6b7280;
}

div[data-testid="stMetricvalue"]{
font-size:34px;
font-weight:800;
color:#111827;
}

/* CARDS */

.ea-card{
background:white;
border-radius:22px;
padding:26px;
box-shadow:
0px 4px 18px rgba(15,23,42,0.05);
margin-bottom:24px;
}

/* TÍTULOS */

.ea-section-title{
font-size:34px;
font-weight:900;
color:#111827;
margin-top:20px;
margin-bottom:20px;
letter-spacing:-1px;
}

/* TABELAS */

div[data-testid="stDataFrame"]{
border-radius:18px;
overflow:hidden;
border:none;
box-shadow:
0px 4px 16px rgba(15,23,42,0.05);
}

/* ALERTAS */

.stAlert{
border-radius:18px;
border:none;
box-shadow:
0px 4px 14px rgba(15,23,42,0.05);
}

/* ABAS */

.stTabs [data-baseweb="tab-list"]{
gap:10px;
}

.stTabs [data-baseweb="tab"]{
background:white;
border-radius:14px 14px 0px 0px;
padding-left:22px;
padding-right:22px;
height:54px;
font-weight:700;
color:#6b7280;
}

.stTabs [aria-selected="true"]{
```

```

        background:#b30000 !important;
        color:white !important;
    }

/* MOBILE */

@media(max-width:900px){
    .ea-hero{
        flex-direction:column;
        align-items:flex-start;
    }

    .ea-hero-left h1{
        font-size:38px;
    }

    .ea-hero-right{
        width:100%;
        flex-direction:column;
    }

    .ea-status-card{
        width:100%;
    }

    .ea-section-title{
        font-size:28px;
    }
}

</style>
,,,,,
unsafe_allow_html=True
)

# =====
# DADOS GOOGLE SHEETS
# =====

url = "https://docs.google.com/spreadsheets/d/1Jqe8xfQfaPEOkHWHuQb7XvQ2aBuuc9UlwBmZ-Lg2jI/export?format=csv"

df = pd.read_csv(url)

# limpar nomes das colunas
df.columns = df.columns.str.strip()

# =====
# PADRONIZAR NOMES DA NOVA PLANILHA
# =====

df.columns = df.columns.str.strip()

df = df.rename(columns={

    "Carimbo de data/hora": "Data",

    # CLIENTE
    "1. Nome do cliente": "Cliente",
    "1. Nome do cliente ": "Cliente",

    # RESPONSÁVEL
    "2. Nome do responsável pela compra": "Responsável Compra",
    "2. Nome do responsável pela compra ": "Responsável Compra",

    # WHATSAPP
    "3. WhatsApp para contato": "WhatsApp",
    "3. WhatsApp para contato ": "WhatsApp",

    # SEGMENTO
    "4. Segmento do cliente": "Segmento",
    "4. Segmento do cliente ": "Segmento",

    # CIDADE
    "5. Cidade": "Cidade",
    "5. Cidade ": "Cidade",

    # BAIRRO
    "6. Bairro": "Bairro",
    "6. Bairro ": "Bairro",

    # ENDEREÇO
    "7. Endereço": "Endereço",
    "7. Endereço ": "Endereço",

    # CEP
    "8. CEP": "CEP",
    "8. CEP ": "CEP",

    # FORNECEDOR
    "9. Quem é o principal fornecedor hoje?": "Concorrente",

```

```

"9. Quem é o principal fornecedor hoje? ": "Concorrente",

# TEMPO
"10. Há quanto tempo trabalha com esse fornecedor?": "Tempo Fornecedor",
"10. Há quanto tempo trabalha com esse fornecedor? ": "Tempo Fornecedor",

# FATOR DECISÃO
"11. O que mais pesa na decisão de compra?": "Fator Decisão",
"11. O que mais pesa na decisão de compra? ": "Fator Decisão",

# FORNECEDOR ATENDE
"12. O fornecedor atual atende totalmente sua necessidade?": "Fornecedor Atende",
"12. O fornecedor atual atende totalmente sua necessidade? ": "Fornecedor Atende",

# DIFICULDADE FORNECEDOR
"13. Qual principal dificuldade com o fornecedor atual?": "Dificuldade Fornecedor Atual",
"13. Qual principal dificuldade com o fornecedor atual? ": "Dificuldade Fornecedor Atual",

# PROBLEMAS
"14. Quais problemas mais acontecem hoje?": "Problemas Atuais",
"14. Quais problemas mais acontecem hoje? ": "Problemas Atuais",

# QUALIDADE
"15. Como você avalia a qualidade do fornecedor atual?": "Qualidade",
"15. Como você avalia a qualidade do fornecedor atual? ": "Qualidade",

# FREQUÊNCIA
"16. Como você avalia a frequência das entregas?": "Frequência Entrega",
"16. Como você avalia a frequência das entregas? ": "Frequência Entrega",

# ATENDIMENTO
"17. Como você avalia o atendimento comercial atual?": "Atendimento Atual",
"17. Como você avalia o atendimento comercial atual? ": "Atendimento Atual",

# JÁ COMPROU PIFPAF
"18. Você já comprou da PifPaf?": "Ja Comprou PifPaf",
"18. Você já comprou da PifPaf? ": "Ja Comprou PifPaf",

# PIFPAF PREÇO
"19. Como você avalia a PifPaf atualmente? [Preço]": "PifPaf Preço",
"19. Como você avalia a PifPaf atualmente? [Preço]": "PifPaf Preço",

# PIFPAF QUALIDADE
"19. Como você avalia a PifPaf atualmente? [Qualidade]": "PifPaf Qualidade",
"19. Como você avalia a PifPaf atualmente? [Qualidade]": "PifPaf Qualidade",

# PIFPAF ENTREGA
"19. Como você avalia a PifPaf atualmente? [Entrega]": "PifPaf Entrega",
"19. Como você avalia a PifPaf atualmente? [Entrega]": "PifPaf Entrega",

# PIFPAF ATENDIMENTO
"19. Como você avalia a PifPaf atualmente? [Atendimento]": "PifPaf Atendimento",
"19. Como você avalia a PifPaf atualmente? [Atendimento]": "PifPaf Atendimento",

# PIFPAF VARIEDADE
"19. Como você avalia a PifPaf atualmente? [Variedade de produtos]": "PifPaf Variedade",
"19. Como você avalia a PifPaf atualmente? [Variedade de produtos]": "PifPaf Variedade",

# PIFPAF NEGOCIAÇÃO
"19. Como você avalia a PifPaf atualmente? [Negociação]": "PifPaf Negociação",
"19. Como você avalia a PifPaf atualmente? [Negociação]": "PifPaf Negociação",

# OPORTUNIDADE PIFPAF
"20. O que faria você comprar mais da PifPaf?": "Oportunidade PifPaf",
"20. O que faria você comprar mais da PifPaf? ": "Oportunidade PifPaf",

# DIFICULDADE PIFPAF
"21. Qual principal dificuldade com a PifPaf hoje?": "Dificuldade PifPaf",
"21. Qual principal dificuldade com a PifPaf hoje? ": "Dificuldade PifPaf",

# CATEGORIA
"22. Qual categoria possui maior volume de vendas?": "Categoria Mais Vendida",
"22. Qual categoria possui maior volume de vendas? ": "Categoria Mais Vendida",

# VOLUME
"23. Qual o volume médio mensal aproximado?": "Volume Mensal",
"23. Qual o volume médio mensal aproximado? ": "Volume Mensal",

# INTERESSE
"Existe interesse em trocar fornecedor?": "Interesse em Trocar",

# POTENCIAL
"Potencial do cliente": "Potencial",

# ESTADO
"Estado": "Estado",

# COORDENADAS
"Latitude": "Latitude",
"Longitude": "Longitude",
"Endereco Completo": "Endereco Completo"
}

```

```
df.columns = df.columns.str.strip()
```

```
# =====  
# COLUNAS PADRÃO  
# =====
```

```
colunas_padrao = {  
    "Data": "",  
    "Cliente": "Não informado",  
    "Responsável Compra": "",  
    "whatsApp": "",  
    "Segmento": "Não informado",  
    "Cidade": "Não informado",  
    "Estado": "MG",  
    "Bairro": "",  
    "Endereço": "",  
    "CEP": "",  
  
    "Concorrente": "Não informado",  
    "Tempo Fornecedor": "",  
    "Fator Decisão": "",  
    "Fornecedor Atende": "",  
    "Dificuldade Fornecedor Atual": "",  
    "Problemas Atuais": "",  
  
    "Qualidade": 0,  
    "Frequência Entrega": 0,  
    "Atendimento Atual": 0,  
  
    "Ja Comprou PifPaf": "",  
    "PifPaf Preço": "",  
    "PifPaf Qualidade": "",  
    "PifPaf Entrega": "",  
    "PifPaf Atendimento": "",  
    "PifPaf Variedade": "",  
    "PifPaf Negociação": "",  
  
    "Oportunidade PifPaf": "",  
    "Dificuldade PifPaf": "",  
  
    "Categoria Mais Vendida": "Não informado",  
    "Volume Mensal": 0,  
    "Potencial": "Baixo",  
    "Interesse em Trocar": "Não",  
  
    "Status Comercial": "Prospectar",  
  
    # compatibilidade com o código antigo  
    "Índice de Atraso": 0,  
    "Itens com Ruptura": 0  
}
```

```
for coluna, valor in colunas_padrao.items():  
    if coluna not in df.columns:  
        df[coluna] = valor
```

```
for coluna, valor in colunas_padrao.items():  
    df[coluna] = df[coluna].replace(["", "nan", "None"], valor)
```

```
# =====  
# CONVERTER TEXTO EM NÚMERO  
# =====
```

```
def converter_volume(valor):  
    valor = str(valor).strip()  
  
    if "Até" in valor:  
        return 2000  
    elif "2.000" in valor and "5.000" in valor:  
        return 5000  
    elif "5.000" in valor and "10.000" in valor:  
        return 10000  
    elif "Acima" in valor:  
        return 15000  
    else:  
        valor = (  
            valor  
            .replace("R$", "")  
            .replace(".", "")  
            .replace(",", ".")  
        )  
        return pd.to_numeric(valor, errors="coerce")
```

```
df["Volume Mensal"] = df["Volume Mensal"].apply(converter_volume).fillna(0)
```

```
colunas_numericas = [  
    "Qualidade",  
    "Frequência Entrega",  
    "Atendimento Atual",  
    "Índice de Atraso",  
    "Itens com Ruptura"  
]
```

```

for coluna in colunas_numericas:
    df[coluna] = pd.to_numeric(df[coluna], errors="coerce").fillna(0)

def converter_nota(valor):
    valor = str(valor).strip().lower()

    if valor in ["ruim", "péssimo", "pessimo"]:
        return 1
    elif valor == "regular":
        return 3
    elif valor == "bom":
        return 4
    elif valor in ["excelente", "ótimo", "otimo"]:
        return 5
    else:
        return pd.to_numeric(valor, errors="coerce")

colunas_pifpaf = [
    "PifPaf Preço",
    "PifPaf Qualidade",
    "PifPaf Entrega",
    "PifPaf Atendimento",
    "PifPaf Variedade",
    "PifPaf Negociação"
]

for coluna in colunas_pifpaf:
    if coluna in df.columns:
        df[coluna] = df[coluna].apply(converter_nota).fillna(0)

# usar frequência como base de atraso operacional
df["Índice de Atraso"] = 5 - df["Frequência Entrega"]

# usar problemas atuais para estimar ruptura
df["Problemas Atuais"] = df["Problemas Atuais"].fillna("").astype(str)

df["Itens com Ruptura"] = df["Problemas Atuais"].apply(
    lambda x: 5 if ("Ruptura" in str(x)) or ("Falta de produtos" in str(x)) else 0
)

# =====
# PADRONIZAR PIFPAF
# =====

df["Concorrente"] = df["Concorrente"].fillna("Não informado").astype(str).str.strip()

df["Concorrente"] = df["Concorrente"].apply(
    lambda x: "PifPaf" if "pif" in x.lower().replace(" ", "") else x
)

# st.success("✅ Dados carregados automaticamente do Google Sheets")

# =====
# TOPO PREMIUM SEM HTML
# =====

st.container(border=True)

col_titulo, col_pifpaf, col_status = st.columns([4,1,1])

with col_titulo:
    st.markdown("""
    # EA Comercial Intelligence

    CRM Analítico Comercial e Operacional

    Região: Vespasiano, Confins e São José da Lapa
    """)

st.success("🟢 Status: Online")

# st.info("Base conectada: Google Forms + Google Sheets")
# =====
# SIDEBAR / FILTROS
# =====

st.sidebar.image(
    "assets/logo_ea.png",
    width=180
)

st.sidebar.caption("CRM Analítico Comercial")

st.sidebar.markdown("---")
st.sidebar.subheader("Filtros Estratégicos")

cidades_lista = sorted(df["Cidade"].dropna().unique())
clientes_lista = sorted(df["Cliente"].dropna().unique())
empresas_lista = sorted(df["Concorrente"].dropna().unique())

```

```

potenciais_lista = sorted(df["Potencial"].dropna().unique())

cidades = st.sidebar.multiselect(
    "Cidade",
    cidades_lista,
    default=cidades_lista
)

clientes = st.sidebar.multiselect(
    "Cliente",
    clientes_lista,
    default=clientes_lista
)

empresas = st.sidebar.multiselect(
    "Empresa / Concorrente",
    empresas_lista,
    default=empresas_lista
)

potenciais = st.sidebar.multiselect(
    "Potencial",
    potenciais_lista,
    default=potenciais_lista
)

st.sidebar.markdown("---")
st.sidebar.caption("Euler Antonio de Souza")
st.sidebar.caption("Status: Online")

df_filtrado = df[
    (df["Cidade"].isin(cidades)) &
    (df["Cliente"].isin(clientes)) &
    (df["Concorrente"].isin(empresas)) &
    (df["Potencial"].isin(potenciais))
]
# =====
# ABAS DO SISTEMA
# =====

aba_geral, aba_operacao, aba_clientes, aba_inteligencia, aba_relatorio, aba_mapa = st.tabs([
    "Visão Geral",
    "Operação",
    "Clientes",
    "Inteligência Comercial",
    "Relatório",
    "Mapa"
])

# =====
# ABA 1 - VISÃO GERAL
# =====

with aba_geral:
    # =====
    # KPIS PREMIUM
    # =====

    st.markdown("## Indicadores Estratégicos")

    total_clientes = len(df_filtrado)

    atraso_medio = round(
        df_filtrado["Índice de Atraso"].mean(),
        1
    )

    ruptura_media = round(
        df_filtrado["Itens com Ruptura"].mean(),
        1
    )

    qualidade_media = round(
        df_filtrado["Qualidade"].mean(),
        1
    )

    clientes_alto = len(
        df_filtrado[
            df_filtrado["Potencial"] == "Alto"
        ]
    )

    col1, col2, col3, col4, col5 = st.columns(5)

    with col1:
        st.info(f"""
        ### Clientes
        # {total_clientes}
        Base total cadastrada
        """)

```



```

with col2:
    st.warning(f"""
    ### Atraso Médio
    # {atraso_medio}
    Performance logística
    """)

with col3:
    st.error(f"""
    ### Ruptura Média
    # {ruptura_media}
    Falhas operacionais
    """)

with col4:
    st.success(f"""
    ### Qualidade
    # {qualidade_media}
    Nível operacional
    """)

with col5:
    st.info(f"""
    ### Alto Potencial
    # {clientes_alto}
    Clientes prioritários
    """)

# =====
# TABELA EXECUTIVA
# =====

st.markdown("---")

st.markdown(
    '<div class="ea-section-title">Dados da Pesquisa</div>',
    unsafe_allow_html=True
)

colunas_tabela = [
    "Data",
    "Cliente",
    "Segmento",
    "Concorrente",
    "Cidade",
    "Índice de Atraso",
    "Itens com Ruptura",
    "Qualidade",
    "Volume Mensal",
    "Potencial", "Oportunidade PifPaf",
    "Dificuldade PifPaf",
    "Problema Operacional",
    "Status Comercial"
]

colunas_existentes = [
    coluna for coluna in colunas_tabela
    if coluna in df_filtrado.columns
]

st.dataframe(
    df_filtrado[colunas_existentes],
    use_container_width=True,
    hide_index=True
)

st.markdown('</div>', unsafe_allow_html=True)

# =====
# ABA 2 – OPERAÇÃO
# =====

with aba_operacao:

    st.markdown(
        '<div class="ea-section-title">Análise visual dos Indicadores</div>',
        unsafe_allow_html=True
    )

    col4, col5 = st.columns(2)

    with col4:

        st.markdown("### Índice de Atraso por Empresa")

        fig_atraso = px.bar(
            df_filtrado,
            x="Concorrente",
            y="Índice de Atraso",
            color="Concorrente",
            text="Índice de Atraso",
            title="Comparativo de Atraso"
        )

```

```

fig_atraso.update_traces(
    textposition="outside"
)

fig_atraso.update_layout(
    height=420,
    showlegend=False,
    xaxis_title="Empresa",
    yaxis_title="Índice de Atraso",
    yaxis=dict(range=[0, 5.5]),
    template="plotly_dark",

    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(0,0,0,0)",

    font=dict(color="white"),
)

st.markdown('<div class="ea-card">', unsafe_allow_html=True)

st.plotly_chart(
    fig_atraso,
    use_container_width=True
)

st.markdown('</div>', unsafe_allow_html=True)

```

with col5:

```

st.markdown("### Qualidade da Entrega")

fig_qualidade = px.bar(
    df_filtrado,
    x="Concorrente",
    y="Qualidade",
    color="Concorrente",
    text="Qualidade",
    title="Comparativo de Qualidade"
)

fig_qualidade.update_traces(
    textposition="outside"
)

fig_qualidade.update_layout(
    height=420,
    showlegend=False,
    xaxis_title="Empresa",
    yaxis_title="Qualidade",
    yaxis=dict(range=[0, 5.5]),
    template="plotly_dark",

    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(0,0,0,0)",

    font=dict(color="white"),
)

st.markdown('<div class="ea-card">', unsafe_allow_html=True)

st.plotly_chart(
    fig_qualidade,
    use_container_width=True
)

st.markdown('</div>', unsafe_allow_html=True)

st.markdown("---")

st.markdown(
    '<div class="ea-section-title">Rupturas por Empresa</div>',
    unsafe_allow_html=True
)

fig_ruptura = px.bar(
    df_filtrado,
    x="Concorrente",
    y="Itens com Ruptura",
    color="Concorrente",
    text="Itens com Ruptura",
    title="Comparativo de Rupturas"
)

fig_ruptura.update_traces(
    textposition="outside"
)

fig_ruptura.update_layout(
    height=420,
    showlegend=False,
    xaxis_title="Empresa",
    yaxis_title="Itens com Ruptura",
    template="plotly_dark",

```

```

        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",

        font=dict(color="white"),
    )

st.markdown('<div class="ea-card">', unsafe_allow_html=True)

st.plotly_chart(
    fig_ruptura,
    use_container_width=True
)

st.markdown('</div>', unsafe_allow_html=True)

st.markdown("---")

st.markdown(
    '<div class="ea-section-title">Comparativo PifPaf x Concorrentes</div>',
    unsafe_allow_html=True
)

df_comparativo = df_filtrado.copy()

df_comparativo["Grupo"] = df_comparativo["Concorrente"].apply(
    lambda x: "PifPaf" if x == "PifPaf" else "Concorrentes"
)

comparativo = df_comparativo.groupby("Grupo").agg({
    "Índice de Atraso": "mean",
    "Qualidade": "mean",
    "Itens com Ruptura": "mean"
}).reset_index()

comparativo_melt = comparativo.melt(
    id_vars="Grupo",
    var_name="Indicador",
    value_name="Resultado"
)

fig_comp = px.bar(
    comparativo_melt,
    x="Indicador",
    y="Resultado",
    color="Grupo",
    barmode="group",
    text="Resultado",
    title="PifPaf x Média dos Concorrentes"
)

fig_comp.update_traces(
    texttemplate="%{text:.2f}",
    textposition="outside"
)

fig_comp.update_layout(
    height=460,
    xaxis_title="Indicador",
    yaxis_title="Resultado Médio",
    template="plotly_dark",

    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(0,0,0,0)",

    font=dict(color="white"),
)

st.markdown('<div class="ea-card">', unsafe_allow_html=True)

st.plotly_chart(
    fig_comp,
    use_container_width=True
)

st.markdown('</div>', unsafe_allow_html=True)

# =====
# ABA - MAPA
# =====
with aba_mapa:

    st.markdown("## Mapa Comercial")

    df_map = df_filtrado.copy()

    if "Latitude" not in df_map.columns:
        df_map["Latitude"] = None

    if "Longitude" not in df_map.columns:
        df_map["Longitude"] = None

    df_map["Latitude"] = (
        df_map["Latitude"]

```

```

        .astype(str)
        .str.replace(",", ".", regex=False)
    )

    df_map["Longitude"] = (
        df_map["Longitude"]
        .astype(str)
        .str.replace(",", ".", regex=False)
    )

    df_map["Latitude"] = pd.to_numeric(
        df_map["Latitude"],
        errors="coerce"
    )

    df_map["Longitude"] = pd.to_numeric(
        df_map["Longitude"],
        errors="coerce"
    )

    df_map = df_map.dropna(
        subset=["Latitude", "Longitude"]
    )

    if df_map.empty:
        st.warning(
            "Nenhuma coordenada encontrada. Rode o script de Latitude/Longitude no Colab antes de usar o mapa."
        )
    else:
        mapa = folium.Map(
            location=[
                df_map["Latitude"].mean(),
                df_map["Longitude"].mean()
            ],
            zoom_start=10,
            tiles="CartoDB positron"
        )

        for _, row in df_map.iterrows():
            atraso = row["Índice de Atraso"]
            ruptura = row["Itens com Ruptura"]
            qualidade = row["Qualidade"]

            risco = "Baixo"
            cor = "green"

            if atraso >= 4 or ruptura >= 5 or qualidade <= 2:
                risco = "Crítico"
                cor = "red"

            elif atraso >= 2 or ruptura >= 3:
                risco = "Atenção"
                cor = "orange"

            folium.CircleMarker(
                location=[
                    row["Latitude"],
                    row["Longitude"]
                ],
                radius=12,
                popup=f"""
                <b>Cliente:</b> {row['Cliente']}<br>
                <b>Cidade:</b> {row['Cidade']}<br>
                <b>Fornecedor:</b> {row['Concorrente']}<br>
                <b>Potencial:</b> {row['Potencial']}<br>
                <b>Risco:</b> {risco}
                """,
                color=cor,
                fill=True,
                fill_color=cor,
                fill_opacity=0.7
            ).add_to(mapa)

        st_folium(
            mapa,
            width=1200,
            height=540
        )

# =====
# ABA 4 - CLIENTES
# =====

with aba_clientes:

    st.markdown(
        '<div class="ea-section-title">Gargalos Detectados</div>',
        unsafe_allow_html=True
    )

```

```

criticos = df_filtrado[
    (df_filtrado["Índice de Atraso"] >= 4) |
    (df_filtrado["Itens com Ruptura"] >= 5)
]

if len(criticos) > 0:
    for _, row in criticos.iterrows():
        st.error(f"""
{row['Cidade']} - {row['Estado']}

Empresa: {row['Concorrente']}

Atraso: {row['Índice de Atraso']}

Rupturas: {row['Itens com Ruptura']}
""")
    else:
        st.success(
            "Nenhum gargalo crítico identificado."
        )
# =====
# RANKING DE QUALIDADE
# =====

st.markdown("---")
st.markdown(
    '<div class="ea-section-title">Ranking de Qualidade</div>',
    unsafe_allow_html=True
)

ranking_qualidade = df_filtrado.sort_values(
    by="Qualidade",
    ascending=False
)

st.dataframe(
    ranking_qualidade[
        [
            "Cliente",
            "Concorrente",
            "Cidade",
            "Qualidade",
            "Potencial"
        ]
    ],
    use_container_width=True,
    hide_index=True
)

# =====
# INSIGHTS
# =====

st.markdown("---")
st.markdown(
    '<div class="ea-section-title">Insights Estratégicos</div>',
    unsafe_allow_html=True
)

melhor_qualidade = df_filtrado.loc[df_filtrado["Qualidade"].idxmax()]
pior_atraso = df_filtrado.loc[df_filtrado["Índice de Atraso"].idxmax()]
maior_ruptura = df_filtrado.loc[df_filtrado["Itens com Ruptura"].idxmax()]

st.success(f"🟢 Melhor qualidade: {melhor_qualidade['Concorrente']}")
st.warning(f"⚠️ Maior atraso: {pior_atraso['Concorrente']}")
st.error(f"🔴 Maior ruptura: {maior_ruptura['Concorrente']}")

# =====
# SCORE COMERCIAL PREMIUM
# =====

st.markdown("---")
st.markdown("## Ranking de Oportunidades")

df_score = df_filtrado[
    df_filtrado["Concorrente"] != "PifPaf"
].copy()

if df_score.empty:
    st.warning("Não existem concorrentes suficientes para gerar ranking de oportunidades.")
else:
    scores = []
    for _, row in df_score.iterrows():

```

```

pontos = 0
pontos += row["Índice de Atraso"] * 10
pontos += row["Itens com Ruptura"] * 6

if row["Qualidade"] <= 2:
    pontos += 20

if row["Potencial"] == "Alto":
    pontos += 30

elif row["Potencial"] == "Médio":
    pontos += 15

if row["Interesse em Trocar"] == "Sim":
    pontos += 35

scores.append(pontos)

df_score["Score Comercial"] = scores

def classificar_prioridade(score):

    if score >= 90:
        return "Alta"

    elif score >= 60:
        return "Média"

    return "Baixa"

df_score["Prioridade"] = df_score["Score Comercial"].apply(classificar_prioridade)

ranking_score = df_score.sort_values(
    by="Score Comercial",
    ascending=False
)

top_oportunidade = ranking_score.iloc[0]

st.markdown("### Principal Oportunidade Comercial")

st.success(f"Cliente: {top_oportunidade['Cliente']}")
st.write(f"Score Comercial: {top_oportunidade['Score Comercial']}")
st.write(f"Prioridade: {top_oportunidade['Prioridade']}")
st.write(f"Concorrente Atual: {top_oportunidade['Concorrente']}")

st.dataframe(
    ranking_score[
        [
            "Cliente",
            "Concorrente",
            "Cidade",
            "Potencial",
            "Score Comercial",
            "Prioridade"
        ]
    ],
    use_container_width=True,
    hide_index=True
)

# =====
# COMPARATIVO PIFPAF
# =====

st.markdown("---")

st.markdown(
    '<div class="ea-section-title">Comparativo PifPaf x Fornecedor Atual</div>',
    unsafe_allow_html=True
)

comparativo_pifpaf = pd.DataFrame({
    "Indicador": [
        "Qualidade",
        "Entrega",
        "Atendimento"
    ],
    "Fornecedor Atual": [
        df_filtrado["Qualidade"].mean(),
        df_filtrado["Frequência Entrega"].mean(),
        df_filtrado["Atendimento Atual"].mean()
    ],
    "PifPaf": [
        df_filtrado["PifPaf Qualidade"].mean(),
        df_filtrado["PifPaf Entrega"].mean(),
        df_filtrado["PifPaf Atendimento"].mean()
    ]
})

comparativo_melt = comparativo_pifpaf.melt(
    id_vars="Indicador",

```

```

var_name="Grupo",
value_name="Nota"
)

fig_comp = px.bar(
    comparativo_melt,
    x="Indicador",
    y="Nota",
    color="Grupo",
    barmode="group",
    text="Nota",
    title="PifPaf x Fornecedor Atual"
)

fig_comp.update_traces(
    texttemplate="%{text:.2f}",
    textposition="outside"
)

fig_comp.update_layout(
    height=460,
    yaxis=dict(range=[0, 5]),
    template="plotly_dark"
)

st.plotly_chart(
    fig_comp,
    use_container_width=True
)

# =====
# OPORTUNIDADES E AÇÕES RECOMENDADAS
# =====

st.markdown("---")

st.markdown(
    '<div class="ea-section-title">Oportunidades e Ações Recomendadas</div>',
    unsafe_allow_html=True
)

for _, row in df_filtrado.iterrows():
    cliente = row["Cliente"]
    empresa = row["Concorrente"]
    atraso = row["Índice de Atraso"]
    ruptura = row["Itens com Ruptura"]
    qualidade = row["Qualidade"]
    potencial = row["Potencial"]

    categoria = (
        row["Categoria Mais Vendida"]
        if "Categoria Mais Vendida" in df_filtrado.columns
        else "categoria Indicadores Operacionais"
    )

    if empresa == "PifPaf":
        if atraso >= 4 or ruptura >= 5 or qualidade <= 2:
            st.error(f"""
Cliente: {cliente}

Gargalo interno identificado na operação da PifPaf.

Atraso: {atraso}
Rupturas: {ruptura}
Qualidade: {qualidade}

Ações recomendadas:
- Revisar rota de entrega
- Avaliar aumento da frequência de abastecimento
- Verificar cadeia fria e conservação
- Reduzir ruptura no cliente
- Criar plano de acompanhamento comercial prioritário
""")

        else:
            st.success(f"""
Cliente: {cliente}

A operação da PifPaf apresenta estabilidade operacional.

Ações recomendadas:
- Manter nível de serviço
- Preservar relacionamento comercial
- Monitorar indicadores preventivamente
""")

    else:
        if atraso >= 4 or ruptura >= 5 or qualidade <= 2:

```

```

        st.warning(f"""
Cliente: {cliente}

Concorrente atual: {empresa}

Falhas operacionais identificadas no fornecedor atual.

Atraso: {atraso}
Rupturas: {ruptura}
Qualidade: {qualidade}

Oportunidade para PifPaf:
- Realizar visita comercial
- Apresentar proposta PifPaf
- Destacar estabilidade logística
- Oferecer mix estratégico de {categoria}
- Usar falhas do concorrente como argumento comercial
""")

    elif potencial == "Alto":

        st.info(f"""
Cliente: {cliente}

Concorrente atual: {empresa}

Cliente de alto potencial, porém com baixa vulnerabilidade operacional no momento.

Ações recomendadas:
- Manter relacionamento ativo
- Monitorar fornecedor atual
- Mapear oportunidades futuras
- Acompanhar volume e categoria comprada
""")

    else:

        st.success(f"""
Cliente: {cliente}

Concorrente atual: {empresa}

Fornecedor atual apresenta estabilidade operacional.

Ação recomendada:
- Monitorar periodicamente
- Manter cliente no radar comercial
""")

```

```

# =====
# ANÁLISE ESTRATÉGICA AVANÇADA
# =====

```

```

st.markdown("---")

st.markdown(
    '<div class="ea-section-title">Análise Estratégica Avançada</div>',
    unsafe_allow_html=True
)

```

```

df_pifpaf = df_filtrado[
    df_filtrado["Concorrente"] == "PifPaf"
]

```

```

df_concorrentes = df_filtrado[
    df_filtrado["Concorrente"] != "PifPaf"
]

```

```

analises = []

```

```

if not df_pifpaf.empty and not df_concorrentes.empty:

```

```

    atraso_pifpaf = df_pifpaf["Índice de Atraso"].mean()
    atraso_concorrente = df_concorrentes["Índice de Atraso"].mean()

```

```

    qualidade_pifpaf = df_pifpaf["Qualidade"].mean()
    qualidade_concorrente = df_concorrentes["Qualidade"].mean()

```

```

    ruptura_pifpaf = df_pifpaf["Itens com Ruptura"].mean()
    ruptura_concorrente = df_concorrentes["Itens com Ruptura"].mean()

```

```

    volume_total = df_filtrado["Volume Mensal"].sum()

```

```

    clientes_prioritarios = len(
        df_filtrado[df_filtrado["Potencial"] == "Alto"]
    )

```

```

    interesse_troca = len(
        df_filtrado[df_filtrado["Interesse em Trocar"] == "Sim"]
    )

```

```

    if atraso_pifpaf > atraso_concorrente:

```



```

        analises.append(
            "A PifPaf apresenta atraso médio acima dos concorrentes. "
            "Isso indica necessidade de revisar rotas, frequência de entrega "
            "e capacidade operacional."
        )
    elif atraso_pifpaf == atraso_concorrente:
        analises.append(
            "A PifPaf apresenta atraso equivalente aos concorrentes. "
            "O desempenho logístico está no mesmo nível do mercado analisado."
        )
    else:
        analises.append(
            "A PifPaf apresenta melhor desempenho em atraso frente aos concorrentes."
        )
    if qualidade_pifpaf < qualidade_concorrente:
        analises.append(
            "Os concorrentes apresentam melhor percepção de qualidade. "
            "Isso indica oportunidade de reforçar atendimento, conservação "
            "e experiência de entrega da PifPaf."
        )
    elif qualidade_pifpaf == qualidade_concorrente:
        analises.append(
            "A PifPaf apresenta qualidade equivalente aos concorrentes."
        )
    else:
        analises.append(
            "A PifPaf demonstra qualidade operacional superior ao mercado analisado."
        )
    if ruptura_pifpaf > ruptura_concorrente:
        analises.append(
            "A ruptura da PifPaf está acima da média dos concorrentes. "
            "Esse ponto representa risco de perda comercial e exige reforço no abastecimento."
        )
    elif ruptura_pifpaf == ruptura_concorrente:
        analises.append(
            "A PifPaf apresenta nível de ruptura equivalente aos concorrentes."
        )
    else:
        analises.append(
            "A PifPaf apresenta melhor estabilidade de abastecimento frente aos concorrentes."
        )
    if clientes_prioritarios > 0:
        analises.append(
            f"Existem {clientes_prioritarios} clientes de alto potencial que devem receber atenção comercial prioritária."
        )
    if interesse_troca > 0:
        analises.append(
            f"Foram identificados {interesse_troca} clientes com abertura para mudança ou ampliação de fornecimento."
        )
    if volume_total >= 10000:
        analises.append(
            "O volume mensal analisado demonstra potencial relevante para crescimento regional."
        )
    analises.append(
        "O cenário geral indica oportunidades de melhoria operacional, fortalecimento logístico "
        "e expansão comercial da PifPaf na região analisada."
    )
    for texto in analises:
        st.info(texto)
else:
    st.warning(
        "Adicione registros da PifPaf e concorrentes para gerar análises comparativas."
    )

```

```
# =====  
# ABA - INTELIGÊNCIA COMERCIAL  
# =====
```

```
with aba_inteligencia:
```

```
    st.markdown("## Inteligência Comercial")  
  
    # =====  
    # SCORE DE CONVERSÃO  
    # =====  
  
    df_score = df_filtrado.copy()  
  
    scores = []  
  
    for _, row in df_score.iterrows():  
        pontos = 0  
  
        if str(row["Interesse em Trocar"]).lower() in ["sim", "talvez"]:  
            pontos += 40  
  
        if str(row["Potencial"]).lower() == "alto":  
            pontos += 35  
  
        elif str(row["Potencial"]).lower() in ["médio", "medio"]:  
            pontos += 20  
  
        problemas = str(row["Problemas Atuais"])  
  
        if "Atrasos" in problemas:  
            pontos += 10  
  
        if "Ruptura" in problemas or "Falta de produtos" in problemas:  
            pontos += 10  
  
        if "Falta de atendimento" in problemas:  
            pontos += 10  
  
        scores.append(pontos)  
  
    df_score["Score Conversão"] = scores  
  
    ranking = df_score.sort_values(  
        by="Score Conversão",  
        ascending=False  
    )  
  
    # =====  
    # FATORES DE DECISÃO  
    # =====
```

```
    st.markdown("---")  
    st.subheader("Fatores de Decisão de Compra")
```

```
    fatores = (  
        df_filtrado["Fator Decisão"]  
        .astype(str)  
        .str.split(",")  
        .explode()  
        .str.strip()  
        .value_counts()  
        .reset_index()  
    )  
  
    fatores.columns = ["Fator", "Quantidade"]  
  
    fig_fatores = px.bar(  
        fatores,  
        x="Fator",  
        y="Quantidade",  
        color="Quantidade",  
        template="plotly_dark",  
        title="O que mais pesa na decisão de compra"  
    )
```

```
    st.plotly_chart(fig_fatores, use_container_width=True)
```

```
    # =====  
    # PROBLEMAS DOS CONCORRENTES  
    # =====
```

```
    st.markdown("---")  
    st.subheader("Principais Problemas dos Concorrentes")
```

```
    problemas = (  
        df_filtrado["Problemas Atuais"]  
        .astype(str)  
        .str.split(",")  
        .explode()  
        .str.strip()  
    )
```

```

        .value_counts()
        .reset_index()
    )

    problemas.columns = ["Problema", "Quantidade"]

    fig_problemas = px.bar(
        problemas,
        x="Problema",
        y="Quantidade",
        color="Quantidade",
        template="plotly_dark",
        title="Problemas identificados nos fornecedores"
    )

    st.plotly_chart(fig_problemas, use_container_width=True)

    # =====
    # OPORTUNIDADES PIFPAF
    # =====

st.markdown("---")
st.subheader("O que faria comprar mais da PifPaf")

oportunidades = (
    df_filtrado["Oportunidade PifPaf"]
    .astype(str)
    .str.split(",")
    .explode()
    .str.strip()
    .value_counts()
    .reset_index()
)

oportunidades.columns = ["Oportunidade", "Quantidade"]

fig_oportunidades = px.bar(
    oportunidades,
    x="Oportunidade",
    y="Quantidade",
    color="Quantidade",
    template="plotly_dark",
    title="Oportunidades de crescimento PifPaf"
)

st.plotly_chart(fig_oportunidades, use_container_width=True)

# =====
# DIFICULDADES PIFPAF
# =====

st.markdown("---")
st.subheader("Principais Dificuldades com a PifPaf")

dificuldades = (
    df_filtrado["Dificuldade PifPaf"]
    .astype(str)
    .str.split(",")
    .explode()
    .str.strip()
    .value_counts()
    .reset_index()
)

dificuldades.columns = ["Dificuldade", "Quantidade"]

fig_dificuldades = px.bar(
    dificuldades,
    x="Dificuldade",
    y="Quantidade",
    color="Quantidade",
    template="plotly_dark",
    title="Principais dificuldades com a PifPaf"
)

st.plotly_chart(fig_dificuldades, use_container_width=True)

# =====
# PARTICIPAÇÃO DOS CONCORRENTES
# =====

st.markdown("---")
st.subheader("Participação dos Concorrentes")

ranking_concorrentes = (
    df_filtrado["Concorrente"]
    .value_counts()
    .reset_index()
)

ranking_concorrentes.columns = ["Concorrente", "Clientes"]

fig_concorrentes = px.pie(

```

```

        ranking_concorrentes,
        names="Concorrente",
        values="Clientes",
        template="plotly_dark",
        title="Participação dos fornecedores"
    )

st.plotly_chart(fig_concorrentes, use_container_width=True)

# =====
# POTENCIAL POR CIDADE
# =====

st.markdown("---")
st.subheader("Potencial Comercial por Cidade")

potencial_cidade = (
    df_filtrado.groupby("Cidade")["Cliente"]
    .count()
    .reset_index()
)

potencial_cidade.columns = ["Cidade", "Clientes"]

fig_cidade = px.bar(
    potencial_cidade,
    x="Cidade",
    y="Clientes",
    color="Clientes",
    template="plotly_dark",
    title="Clientes mapeados por cidade"
)

st.plotly_chart(fig_cidade, use_container_width=True)

# =====
# RADAR – PERCEPÇÃO PIFPAF
# =====

st.markdown("---")
st.subheader("Radar de Percepção da PifPaf")

colunas_radar = [
    "PifPaf Preço",
    "PifPaf Qualidade",
    "PifPaf Entrega",
    "PifPaf Atendimento",
    "PifPaf Variedade",
    "PifPaf Negociação"
]

colunas_existentes_radar = [
    coluna for coluna in colunas_radar
    if coluna in df_filtrado.columns
]

if len(colunas_existentes_radar) > 0:
    df_radar = df_filtrado[colunas_existentes_radar].copy()

    for coluna in colunas_existentes_radar:
        df_radar[coluna] = pd.to_numeric(
            df_radar[coluna],
            errors="coerce"
        ).fillna(0)

    radar_media = df_radar.mean().reset_index()
    radar_media.columns = ["Indicador", "Nota"]

    fig_radar = px.line_polar(
        radar_media,
        r="Nota",
        theta="Indicador",
        line_close=True,
        template="plotly_dark",
        title="Percepção média da PifPaf"
    )

    fig_radar.update_traces(fill="toself")

    st.plotly_chart(fig_radar, use_container_width=True)

# =====
# HEATMAP
# =====

st.markdown("---")
st.subheader("Mapa de Força Comercial por Cidade e Concorrente")

heatmap = (
    df_filtrado
    .groupby(["Cidade", "Concorrente"])["Cliente"]
    .count()

```

```

    .reset_index()
)

heatmap.columns = ["Cidade", "Concorrente", "Clientes"]

fig_heatmap = px.density_heatmap(
    heatmap,
    x="Concorrente",
    y="Cidade",
    z="Clientes",
    template="plotly_dark",
    title="Concentração de clientes por cidade e fornecedor"
)

st.plotly_chart(fig_heatmap, use_container_width=True)

# =====
# FUNIL COMERCIAL
# =====

st.markdown("---")
st.subheader("Funil Comercial")

total_mapeados = len(df_filtrado)

interessados = len(
    df_filtrado[
        df_filtrado["Interesse em Trocar"]
        .astype(str)
        .str.contains("Sim|Talvez", case=False, na=False)
    ]
)

alto_potencial = len(
    df_filtrado[
        df_filtrado["Potencial"]
        .astype(str)
        .str.contains("Alto", case=False, na=False)
    ]
)

oportunidades_qtd = len(
    df_filtrado[
        df_filtrado["Oportunidade PifPaf"].astype(str).str.len() > 3
    ]
)

funil = pd.DataFrame({
    "Etapa": [
        "Clientes mapeados",
        "Com abertura",
        "Alto potencial",
        "Oportunidade identificada"
    ],
    "Quantidade": [
        total_mapeados,
        interessados,
        alto_potencial,
        oportunidades_qtd
    ]
})

fig_funil = px.funnel(
    funil,
    x="Quantidade",
    y="Etapa",
    template="plotly_dark",
    title="Funil de oportunidades comerciais"
)

st.plotly_chart(fig_funil, use_container_width=True)

# =====
# MATRIZ DE OPORTUNIDADE
# =====

st.markdown("---")
st.subheader("Matriz de Oportunidade Comercial")

fig_bubble = px.scatter(
    ranking,
    x="Volume Mensal",
    y="Score Conversão",
    size="Score Conversão",
    color="Concorrente",
    hover_name="Cliente",
    template="plotly_dark",
    title="Volume mensal x Score de conversão"
)

st.plotly_chart(fig_bubble, use_container_width=True)

# =====

```

```

# RANKING DE CONVERSÃO
# =====

st.markdown("---")
st.subheader("Ranking de Conversão Comercial")

st.dataframe(
    ranking[
        [
            "Cliente",
            "Cidade",
            "Concorrente",
            "Potencial",
            "Interesse em Trocar",
            "Score Conversão"
        ]
    ],
    use_container_width=True,
    hide_index=True
)

# =====
# ABA 5 - RELATÓRIO
# =====

with aba_relatorio:

    st.markdown(
        '<div class="ea-section-title">Relatório Executivo</div>',
        unsafe_allow_html=True
    )

    if st.button("Gerar Relatório PDF"):

        media_atraso = df_filtrado["Índice de Atraso"].mean()
        media_qualidade = df_filtrado["Qualidade"].mean()
        total_ruptura = df_filtrado["Itens com Ruptura"].sum()
        volume_total = (
            df_filtrado["Volume Mensal"].sum()
            if "Volume Mensal" in df_filtrado.columns
            else 0
        )

        # =====
        # GRÁFICO PARA PDF
        # =====

        fig_pdf, ax_pdf = plt.subplots(
            figsize=(8, 4)
        )

        ranking_plot = (
            df_filtrado
            .groupby("Concorrente")["Índice de Atraso"]
            .mean()
        )

        ranking_plot.plot(
            kind="bar",
            ax=ax_pdf
        )

        plt.title("Média de Atraso por Empresa")
        plt.tight_layout()
        plt.savefig("grafico_atraso.png")
        plt.close()

        # =====
        # DOCUMENTO PDF
        # =====

        pdf = SimpleDocTemplate(
            "relatorio_executivo.pdf",
            pagesize=A4,
            rightMargin=30,
            leftMargin=30,
            topMargin=30,
            bottomMargin=30
        )

        elementos = []
        estilos = getSampleStyleSheet()

        titulo = Paragraph(
            """
            <font size=20>
            <b>RELATÓRIO EXECUTIVO - EA INTELLIGENCE</b>
            </font>
            """
            , estilos["Title"]
        )

        elementos.append(titulo)
        elementos.append(Spacer(1, 20))

```

```

subtitulo = Paragraph(
    """
    EA Comercial Intelligence<br/>
    CRM Analítico Comercial e Operacional
    """
    estilos["BodyText"]
)

elementos.append(subtitulo)
elementos.append(Spacer(1, 25))

# =====
# RESUMO EXECUTIVO
# =====

elementos.append(
    Paragraph(
        "<b>Resumo Executivo</b>",
        estilos["Heading2"]
    )
)

elementos.append(Spacer(1, 10))

if "analises" in globals() and len(analises) > 0:
    resumo_texto = "<br/><br/>".join(analises)
else:
    resumo_texto = (
        "Relatório gerado automaticamente com base nos dados filtrados "
        "do dashboard EA Intelligence."
    )

elementos.append(
    Paragraph(
        resumo_texto,
        estilos["BodyText"]
    )
)

elementos.append(Spacer(1, 20))

# =====
# KPIS DO RELATÓRIO
# =====

elementos.append(
    Paragraph(
        "<b>Indicadores Estratégicos</b>",
        estilos["Heading2"]
    )
)

elementos.append(Spacer(1, 10))

total_clientes = len(df_filtrado)
atraso_medio = df_filtrado["Índice de Atraso"].mean()
ruptura_media = df_filtrado["Itens com Ruptura"].mean()
qualidade_media = df_filtrado["Qualidade"].mean()

clientes_alto = len(
    df_filtrado[df_filtrado["Potencial"] == "Alto"]
)

dados_kpi = [
    ["Indicador", "Resultado"],
    ["Total de Clientes", total_clientes],
    ["Atraso Médio", round(atraso_medio, 2)],
    ["Ruptura Média", round(ruptura_media, 2)],
    ["Qualidade Média", round(qualidade_media, 2)],
    ["Clientes Alto Potencial", clientes_alto],
    ["Volume Mensal", f"R$ {volume_total:,.0f}"]
]

tabela_kpi = Table(
    dados_kpi,
    colwidths=[220, 220]
)

tabela_kpi.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#991b1b")),
    ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
    ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
    ("GRID", (0, 0), (-1, -1), 1, colors.black),
    ("BACKGROUND", (0, 1), (-1, -1), colors.whitesmoke),
    ("FONTSIZE", (0, 0), (-1, -1), 9)
]))

elementos.append(tabela_kpi)
elementos.append(Spacer(1, 20))

```

```

# =====
# GRÁFICO
# =====

elementos.append(
    Paragraph(
        "<b>Análise Gráfica</b>",
        estilos["Heading2"]
    )
)

elementos.append(Spacer(1, 10))

elementos.append(
    Image(
        "grafico_atraso.png",
        width=450,
        height=250
    )
)

elementos.append(Spacer(1, 20))

# =====
# TABELA DE CLIENTES
# =====

elementos.append(
    Paragraph(
        "<b>Clientes Prioritários</b>",
        estilos["Heading2"]
    )
)

elementos.append(Spacer(1, 10))

colunas_pdf = [
    "Cliente",
    "Concorrente",
    "Cidade",
    "Índice de Atraso",
    "Itens com Ruptura",
    "Qualidade",
    "Potencial"
]

colunas_pdf_existentes = [
    coluna for coluna in colunas_pdf
    if coluna in df_filtrado.columns
]

tabela_pdf = df_filtrado[colunas_pdf_existentes]

dados_tabela = [
    tabela_pdf.columns.tolist()
] + tabela_pdf.values.tolist()

tabela = Table(
    dados_tabela,
    repeatRows=1
)

tabela.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#991b1b")),
    ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
    ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
    ("FONTSIZE", (0, 0), (-1, -1), 8),
    ("GRID", (0, 0), (-1, -1), 1, colors.black),
    ("BACKGROUND", (0, 1), (-1, -1), colors.beige)
]))

elementos.append(tabela)
elementos.append(Spacer(1, 20))

# =====
# CONCLUSÃO
# =====

elementos.append(
    Paragraph(
        "<b>Conclusão Estratégica</b>",
        estilos["Heading2"]
    )
)

elementos.append(Spacer(1, 10))

conclusao = Paragraph(
    "O cenário analisado demonstra oportunidades relevantes para fortalecimento operacional, expansão comercial e aumento da competitividade da PifPaf frente aos concorrentes avaliados."
)

```



```
        estilos["BodyText"]
    )
    elementos.append(conclusao)
    pdf.build(elementos)
    with open("relatorio_executivo.pdf", "rb") as arquivo:
        st.download_button(
            label="Baixar Relatório Executivo",
            data=arquivo,
            file_name="relatorio_executivo.pdf",
            mime="application/pdf"
        )
    )
```